

Managing Lift's for Buildings .

Problem Overview : This system models a lift control system for a building. It manages floor requests, assigns lifts using a controller with the help of Scheduler, and controls lift movement efficiently.

Flow of How Lift's working:

Floor Button Request

| (requests)

LiftController Decision

|

SchedulerStrategy Assistance

| (help to take decision for Controller)

Lift Execution

| (commands)

Motor Movements (and other actuators work)

Entity Identification:

1. Entity : Building
 - Description : representation of numbers of Floor's and Lift's.
 - Attributes : B_id, B_no, _no_of_floors, no_of_lifts
 - Responsibilities :
 - Maintains structural composition of floors and lifts
 - Initializes lift system components
 - Provides configuration parameters to controller
2. Entity : Floor
 - Description : Represents a physical floor in the building where users can request lift service.
 - Attributes : floorNumber
 - Responsibilities :
 - Represents the physical floor location
 - Provides interface for lift request through buttons
 - Displays the floor number
3. Entity : FloorButton
 - Description : Represents a directional button on a floor that captures user intent to move up or down.
 - Attributes : direction(up/down), ispressed
 - Responsibilities :
 - Captures user input
 - Generates lift request based on directiona and Notifies the LiftController
 - Displays pressed state

4. Entity : LiftController

- Description : Acts as the central coordination component that receives lift requests from floors and assigns them to appropriate lifts using a scheduling strategy.
- Attributes : liftsStatusMap, reuestQueue, schedulerStrategy
- Responsibilities:
 - Receives lift requests from FloorButtons
 - Maintains real-time status of all lifts
 - Invokes the scheduling strategy to select the optimal lift
 - Assigns requests to selected lifts
 - Updates lift stop lists accordingly
- Component Name : SchedulerStrategy
 - Description : Defines a strategy for selecting the most suitable lift for servicing a given request based on current lift states.
 - Responsibilities:
 - Evaluates the current state of all lifts (position, direction, load)
 - Analyzes an incoming lift request (floor number and direction)
 - Computes priority or cost for each lift
 - Selects and returns the optimal lift to the controller

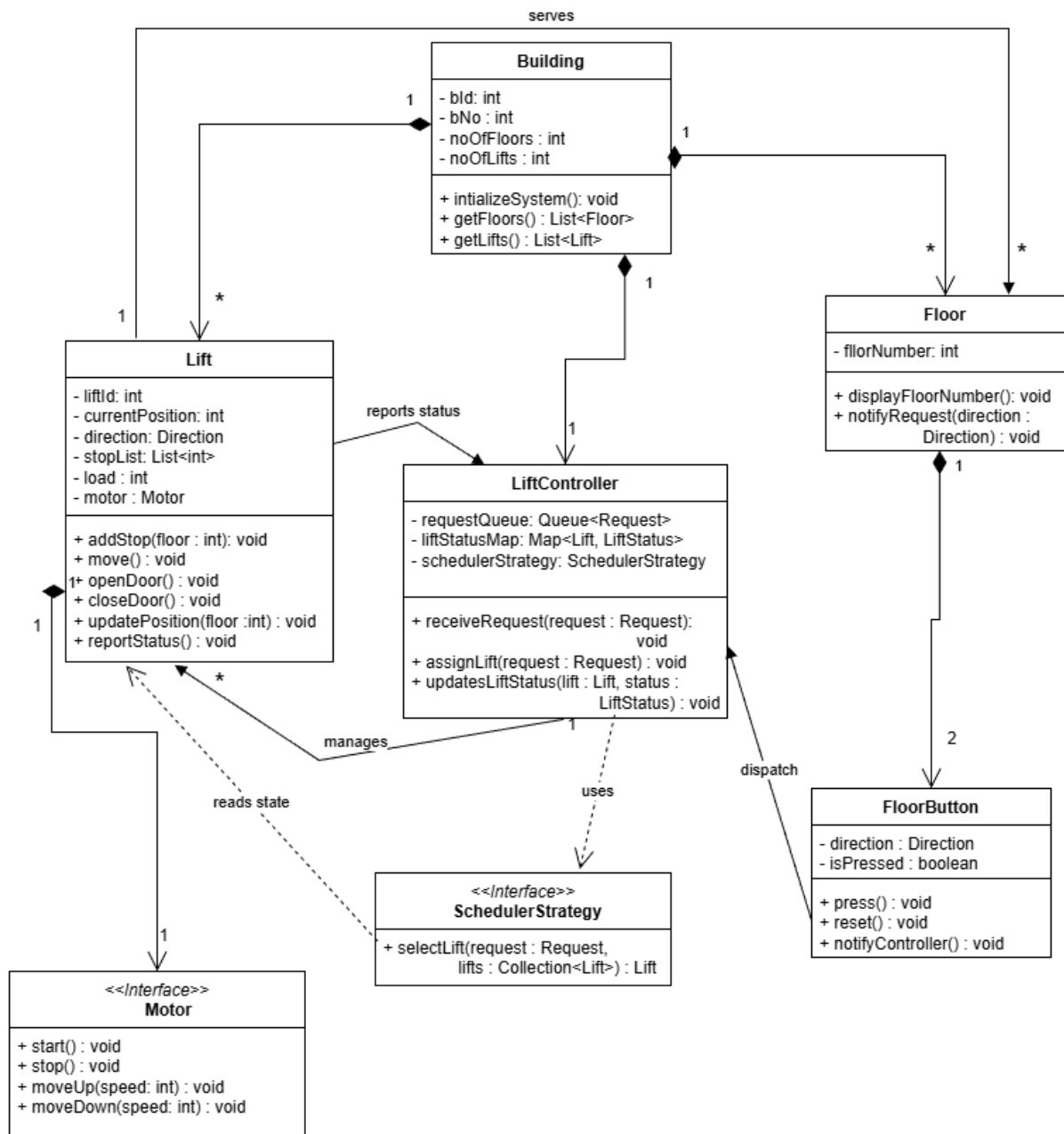
5. Entity : Lift

- Description : Represents the lift car responsible for transporting users between floors.
It maintains its movement state and executes commands assigned by the LiftController.
- Attributes : liftId, currPosition directions(up/down/IDLE), stopList, load, Motor.
- Responsibilities :
 - Maintains its current state (floor, direction, load)
 - Receives assigned requests from the LiftController
 - Manages and processes the stop list
 - Moves between floors according to assigned stops
 - Opens and closes doors at designated floors
 - Sends status updates back to the controller
- Component Name : Motor
 - Description : Represents the physical actuator responsible for moving the lift car vertically.It executes movement commands issued by the Lift entity.
 - Responsibilities :
 - Executes movement commands received from the Lift
 - Moves the lift car up or down based on direction
 - Adjusts speed as instructed
 - Stops movement when commanded or when a floor is reached

Relationships between Entity and Component :

1. Building → Floor **Relationship** : Composition (HAS-A)
(Building contains multiple Floors)
2. Building → Lift **Relationship** : Composition (HAS-A)
(Building contain multiple Lifts)
3. Building → LiftController **Relationship** : Composition and Associations
(Building initializes and owns LiftController)
4. Floor → FloorButton **Relationship** : Composition(Has-A)
(Floor has FloorButtons (UP and/or DOWN))
5. FloorButton → LiftController **Relationship** : Association(uses)
(FloorButtons send Request to LiftController)
6. LiftController → Lift **Relationship** : Association (Manages)
(LiftController manages multiple Lifts)
7. LiftController → SchedulerStrategy **Relationship** : Dependency (Uses Strategy)
8. SchedulerStrategy → Lift **Relationship**: Dependency (Reads State)
(The scheduler reads lift position, direction, and load to decide which lift is optimal)
9. Lift → Motor **Relationship**: Composition (HAS-A)
(The motor is a physical component of the lift and cannot exist independently)
10. Lift → Floor **Relationship**: Association (Serves)
(Lift serves multiple Floors)
11. Lift → LiftController **Relationship**: Association (Reports Status)
(Lift reports status to LiftController)

Class Diagram :



Detailed System Working :

1. User Request Generation

- A user presses the **UP or DOWN button** on a floor.
- The FloorButton captures the user's intent (direction).
- The button generates a lift request and sends it to the LiftController.

2. Request Handling by LiftController

- The LiftController receives the request and stores it in a request queue.
- It maintains the **real-time status** of all lifts, including their current position, direction, and load.

3. Lift Selection Using SchedulerStrategy

- The LiftController delegates the decision of lift selection to the SchedulerStrategy.
- The scheduler evaluates all available lifts by reading their current state.
- Based on predefined criteria (such as distance, direction, and load), the scheduler selects the **optimal lift**.
- The selected lift is returned to the controller.

4. Lift Execution

- The LiftController assigns the request to the selected Lift.
- The lift updates its internal stopList and determines the next target floor.
- The lift decides the movement direction based on its current position and target floor.

5. Motor Movement and Physical Execution

- The Lift sends movement commands to the Motor.
- The motor executes these commands to move the lift car up or down.
- When the target floor is reached, the lift stops, opens doors, and allows passengers to enter or exit.

6. Status Reporting and Continuous Operation

- After completing each movement, the lift reports its updated status back to the LiftController.
- The controller updates its lift status map and continues processing pending requests.
- This cycle repeats continuously, enabling efficient and coordinated lift operation.